

# Homework: Wealth Calculator System

*Java 25 • Classes • Records • Composition • Dependency Injection • Maven • JUnit*

In this homework you will complete a small wealth-calculation system. The goal is not only to produce correct numerical results, but also to practice designing small Java classes that have clear responsibilities and collaborate through constructor-based dependency injection. You must do this homework independently.

**Due date: Monday, 9/14, by the end of the day.**

## Learning Objectives

- Design concrete Java classes with focused responsibilities.
- Use composition and constructor-based dependency injection.
- Use Java records to represent immutable input and result data.
- Use collections returned by provided classes.
- Separate calculation logic from reporting and file-reading responsibilities.
- Use a Java 25 compact source file as the application's composition root.
- Use Maven and instructor-provided JUnit 5 tests to check your implementation.

## Starter Project

A Maven starter project is provided. Do not rename the required classes, records, constructors, or public methods because the instructor-provided tests depend on this API.

| Provided by Instructor | Student Completes             |
|------------------------|-------------------------------|
| pom.xml                | FixedWealthCalculator.java    |
| RateFileReader.java    | VariableWealthCalculator.java |
| SavingsFileReader.java | ConsoleReporter.java          |
| rates.txt              | Main.java                     |
| savings.txt            | Required records/API          |
| JUnit 5 tests          |                               |

## Input Assumptions

You may assume that all input is correct. Input validation and error handling are not part of this homework.

- Files exist and contain correctly formatted numeric values.
- Rates and savings files contain the expected number of entries.
- Numeric calculation inputs are valid.
- You do not need to handle malformed, missing, or empty files.

## Required Financial Calculations

Your system must implement five calculations.

### 1. Future Value - Fixed Lump Sum

Calculate the future value of an initial amount using a constant annual interest rate.

$$FV = PV(1 + r)^n$$

### 2. Compound Savings - Fixed Contribution and Rate

Calculate the future value of a starting balance with the same savings contribution made at the end of each year and a constant annual interest rate.

```
for each year:
    balance = balance * (1 + rate)
    balance = balance + contribution
```

### 3. Present Value - Fixed Rate

Calculate the present value of a future amount using a constant annual interest rate.

$$PV = FV / (1 + r)^n$$

### 4. Future Value - Variable Annual Rates

Use the annual interest rates supplied by RateFileReader. Apply each rate, in order, to the current balance.

```
for each rate:
    balance = balance * (1 + rate)
```

### 5. Compound Savings - Variable Rates and Contributions

Use the rates supplied by RateFileReader and the annual contributions supplied by SavingsFileReader. Savings are deposited at the end of each year.

```
for each year:
    balance = balance * (1 + rate)
    balance = balance + contribution
```

## Required Design

### FixedWealthCalculator

This class performs the three calculations that use fixed values. It must not read files or print output.

```
public class FixedWealthCalculator {
    public CalculationResult futureValue(LumpSumInput input) { ... }

    public CalculationResult compoundSavings(SavingsInput input) { ... }

    public CalculationResult presentValue(PresentValueInput input) { ... }
}
```

### VariableWealthCalculator

This class performs the two variable-value calculations. The provided file-reader objects are dependencies and must be received through the constructor.

```

public class VariableWealthCalculator {
    private final RateFileReader rateReader;
    private final SavingsFileReader savingsReader;

    public VariableWealthCalculator(
        RateFileReader rateReader,
        SavingsFileReader savingsReader) {
        this.rateReader = rateReader;
        this.savingsReader = savingsReader;
    }

    public CalculationResult futureValue(double startingAmount) { ... }

    public CalculationResult compoundSavings(double startingAmount) { ... }
}

```

**Dependency-injection requirement:** VariableWealthCalculator may not create RateFileReader or SavingsFileReader itself. Those objects must be created elsewhere and passed to its constructor.

## Provided File Readers

You do not implement or modify these classes. Their implementation details are intentionally hidden from the calculator.

```

public class RateFileReader {
    public RateFileReader(String filename) { ... }
    public List<Double> readRates() { ... }
}

public class SavingsFileReader {
    public SavingsFileReader(String filename) { ... }
    public List<Double> readSavings() { ... }
}

```

## ConsoleReporter

ConsoleReporter is responsible only for displaying CalculationResult objects. Calculator classes must return results rather than print them.

```

public class ConsoleReporter {
    public void report(CalculationResult result) {
        // display the result
    }
}

```

Example output:

```
Future Value: $16,288.95
```

## Java Records

Use records to represent calculation input and output data. Records are appropriate here because these objects primarily hold immutable data.

```

public record LumpSumInput(
    double amount,
    double rate,
    int years) {
}

public record SavingsInput(

```

```

        double startingAmount,
        double contribution,
        double rate,
        int years) {
    }

    public record PresentValueInput(
        double futureValue,
        double rate,
        int years) {
    }

    public record CalculationResult(
        String name,
        double value) {
    }

```

## Java 25 Compact Source File

Main.java must use Java 25 compact-source-file syntax. It acts as the composition root: the place where the major objects are created and dependencies are connected. Your Main.java should follow the structure below.

```

void main() {
    var rateReader = new RateFileReader("rates.txt");
    var savingsReader = new SavingsFileReader("savings.txt");

    var fixedCalculator = new FixedWealthCalculator();

    var variableCalculator =
        new VariableWealthCalculator(
            rateReader,
            savingsReader);

    var reporter = new ConsoleReporter();

    // Create inputs, perform all five calculations,
    // and send each result to reporter.report(...).
}

```

Do not place the financial formulas in Main.java. Main should create objects, connect them, call their public methods, and report results.

## What Belongs in Main.java

Main.java should demonstrate the following application flow:

```

Create objects
    ↓
Inject dependencies
    ↓
Create input data
    ↓
Call calculator methods
    ↓
Receive CalculationResult
    ↓
Send result to ConsoleReporter

```

## What Does Not Belong in Main.java

Main.java should not contain business or calculation logic. In particular, do not:

- write financial formulas in Main.java;
- write loops that perform the variable-rate or variable-savings calculations;
- read or parse the contents of the data files directly;
- print calculation results directly with System.out instead of using ConsoleReporter; or
- create RateFileReader or SavingsFileReader inside VariableWealthCalculator.

The financial algorithms belong in the calculator classes. File-reading behavior belongs in the provided reader classes. Output behavior belongs in ConsoleReporter. Main.java connects these objects and uses their public APIs.

## Provided Data Files

The starter project contains the following files.

```
rates.txt
0.05
0.04
0.06
0.035
0.055

savings.txt
5000.00
5500.00
6000.00
6500.00
7000.00
```

## Maven and Instructor-Provided Unit Tests

The starter project contains JUnit 5 tests. You are not required to write the basic unit tests yourself. Use the tests while developing your solution.

From the root directory of the project, run:

```
mvn test
```

Maven will compile your Java source, compile the tests, run them, and report which tests passed or failed. The starter calculator methods contain placeholder results, so some tests are expected to fail until you implement the TODOs.

Before submitting, run:

```
mvn clean test
```

A completed solution should end with:

```
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

## What the Tests Check

- Fixed lump-sum future value.
- Fixed compound savings.
- Present value.
- Variable-rate future value.
- Variable-rate and variable-savings calculation.
- Basic operation of the two provided file-reader classes.

Do not modify, delete, disable, or rename the instructor-provided tests. Passing the tests is necessary, but the design requirements are also part of the assignment.

## Recommended Development Process

1. Open the Maven starter project and confirm that the project SDK and Maven runner use JDK 25.
2. Run `mvn test` and observe the initial results.
3. Review the required records and public APIs.
4. Implement `FixedWealthCalculator` and run `mvn test` again.
5. Implement `VariableWealthCalculator` using the injected file readers and run `mvn test` again.
6. Implement `ConsoleReporter`.
7. Complete `Main.java` using the Java 25 compact source-file syntax.
8. Run `mvn clean test` and confirm BUILD SUCCESS.

## Design Requirements

- Use the required classes, records, constructors, and method signatures.
- Keep calculation logic inside the calculator classes.
- Do not print from either calculator class.
- Do not perform financial calculations in `ConsoleReporter`.
- Use constructor-based dependency injection for `VariableWealthCalculator`.
- Store injected dependencies in private final fields.
- Do not construct file-reader dependencies inside `VariableWealthCalculator`.
- Use the provided file readers rather than rewriting their behavior.
- Use `Main.java` as the composition root rather than as a business-logic class.
- Keep the code readable and appropriately encapsulated.

## What Not to Do

- Do not place all five calculations in `Main.java`.
- Do not combine reporting and calculation logic.
- Do not modify the provided file readers to perform calculations.
- Do not hard-code the expected answers from the unit tests.
- Do not use static mutable fields to share application state.
- Do not modify the instructor-provided tests to make them pass.

## Required README: Design Decisions

Your submission must include a README file. The README is part of the homework and should explain the design decisions in your own solution. Answer each of the five questions below in approximately 2-4 sentences. You do not need to write a formal report.

1. Class responsibilities: What is the responsibility of FixedWealthCalculator, VariableWealthCalculator, and ConsoleReporter? Why are these responsibilities separated into different classes?
2. Dependency injection: What does dependency injection mean in this program? Identify the dependencies that are injected into VariableWealthCalculator.
3. Creating dependencies: Why are RateFileReader and SavingsFileReader created in Main.java and passed to VariableWealthCalculator instead of being created inside VariableWealthCalculator?
4. Records versus classes: Why are LumpSumInput, SavingsInput, PresentValueInput, and CalculationResult records, while FixedWealthCalculator and VariableWealthCalculator are regular classes?
5. Separating calculation from output: Why do the calculator methods return CalculationResult objects instead of printing the answers directly? What design benefit does this provide?

The README will be graded on whether you clearly explain the design of your own program. Short, specific explanations in your own words are required.

Suggested README.md structure:

```
# Wealth Calculator Design Decisions

## 1. Class Responsibilities
...

## 2. Dependency Injection
...

## 3. Creating Dependencies
...

## 4. Records Versus Classes
...

## 5. Separating Calculation from Output
...
```

## Submission

Zip the completed maven project which should include your README file with the design decision questions and submit it to the assignment link in Canvas.

- README.md containing answers to the five required design-decision questions.

## Grading Emphasis

Part of your grade will be a correct solution in which small Java objects have clear responsibilities and are connected through composition and dependency injection that passes the unit tests provided. We will also run

your solution with a different set of input files, your solutions should still work the same. 25% of your grade will be your answers in the README file to the questions above.